Software Project Management Plan Low-Cost Campus ERP System

Project Name: Cloud-Connected Campus ERP & Predictive/Generative Services
Duration: 36 Hours (Hackathon Timeline)
Goal: Implement a cohesive, low-cost ERP solution using ubiquitous cloud office tools (e.g., Google Workspace/Microsoft Office) and integrate Machine Learning (Dropout Prediction) and LLM (AI Counselor) services.

1. Introduction and Scope

1.1 Problem Statement Summary

Public colleges require a unified system for admissions, fees, hostel, and exams, but comprehensive ERPs are too costly. The solution is to leverage familiar, interlinked cloud office suites and lightweight scripting to create a single source of truth and streamline processes.

1.2 Solution Scope

The project focuses on demonstrating seamless data flow and core functionality across four key areas, plus the integration of the specialized ML/LLM microservices.

In-Scope:

1. **Unified Access Layer (Frontend):** Basic student/admin portal UI.
2. **Central Database:** Core data models for Student, Fee, Hostel (via Cloud Sheets/Tables).
3. **Core Workflows:** Admission intake -> Fee Receipting -> Hostel Allocation.
4. **Dashboards:** Summary Admin Dashboard (KPIs).
5. **ML/LLM Integration:** Functional API calls to /predict_risk and /chat from the frontend.

Out-of-Scope (for 36 hours):

1. Full security implementation (Focus on RBAC concept demonstration).
2. Advanced Exam/Library modules (Focus on basic record updates).
3. Production-level deployment/scaling.
4. Full-fledged Payment Gateway integration (Simulation via form submission).

2. Project Team and Responsibilities

The team is structured into three main focus areas with cross-functional support (Navneet, Snigdha, Akash).

Role	Team Member(s)	Key Responsibilities

Frontend/UI (F)	Sandeep, Aryan, Rouashni	Unified Access Portal (Student/Admin UI), Integration of ML/LLM endpoints, UX/Design Polish.
Backend/DB (B)	Snigdha, Aakash Tiwari	Central Database Schema (Sheets/Tables), Admission & Fee Workflow Automation Scripts, Dashboard setup, RBAC simulation.
ML/LLM/DevOps (M)	Navneet Shreya	FastAPI Microservice Integration (API calls), Hostel Allocation Scripting, Notification Scripts, overall system integration glue code.

3. Implementation Plan (36-Hour Breakdown)

The plan follows a time-boxed, phased approach, prioritizing visible progress and essential integration points.

Phase 1: Foundation & UI Mockup (Hours 0-4) - *Focus: Frontend*

Time	Task	Owner	Dependency
0-1	Environment Setup & Final Sync	All	-
1-4	Basic Unified Portal Shell (Login, Navigation)	F	-
0-4	Database Schema Creation (Student, Fee, Hostel Tables)	B, M	-

Phase 2: Core Workflow & Data Flow (Hours 5-16) - *Focus: Backend &*

Scripting

Time	Task	Owner	Dependency
5-8	Admission Form to Database Script	B	P1 DB Schema
8-12	Automated Fee Receipting Script & DB Update	B	P1 DB Schema
8-12	Integrate Frontend to display basic Admission/Fee Status	F	P2 DB Access
13-16	Intelligent Hostel Allocation Script (Priority Assessment & Update)	M	P1 Hostel Table

Phase 3: Intelligence & Dashboards (Hours 17-28) - *Focus: ML/LLM Integration & Visibility*

Time	Task	Owner	Dependency
17-20	Integrate ML Dropout Risk (/predict_risk) into Backend	M	P2 DB Access
20-24	Management Dashboard Creation (Enrollment, Fees, Occupancy KPIs)	B	P2 Core Workflows
24-28	Integrate LLM AI Counselor (/chat) into Student Portal	F	M’s API Endpoint

	UI		
24-28	Basic Exam & Library Record Scripts	B	P2 DB Access

Phase 4: Polish, Security & Demo Prep (Hours 29-36) - *Focus: Integration & Presentation*

Time	Task	Owner	Dependency
29-32	Automated Notification Triggers (Fee Deadline/Allocation)	M	P2 Fee & Hostel Data
29-32	RBAC Simulation/Demo Setup	B	F's Portal Shell
32-34	UI/UX Polish, Responsive Design Review	F	P3 Integration
34-36	Final Integration Testing & Demo Rehearsal	All	All prior phases

4. Quality Management and Testing

4.1 Integration Points Testing (Critical)

Integration	Test Scenario	Owner
Admissions Flow	New application submitted ->StudentMaster updated -> Status visible on Frontend.	F, B

Fee Workflow	Payment submitted -> FeeLedger updated -> Digital Receipt generated.	B, M
ML Integration	Student data passed -> predict_risk returns score -> Score displayed on Admin Dashboard.	M, F
LLM Integration	Student asks a question -> chat returns contextual advice.	F, M

4.2 Code Standards

- **Scripting:** Robust error handling (try-catch), clear comments, and idempotent design where possible.
- **Frontend:** Mobile-responsive design is mandatory (Unified Access Layer).

5. Risk Management

Risk ID	Risk Description	Probability	Impact	Mitigation Strategy	Owner
R1	Cloud Office Scripting API Throttling/Limits	High	Medium	Use robust exponential backoff in scripts; focus on minimizing API calls via bulk operations.	B, M
R2	ML/LLM Service Integration Issues (API Key, Endpoint Failure)	Medium	High	Navneet to ensure endpoints are stable and pre-tested with mock	M

				data before full integration. Implement client-side error fallback messages.	
R3	Data Consistency across tables (Synchronization failure)	Medium	High	Use a single, central script as the **only** write access point to the master data tables to enforce normalization.	B, M
R4	Design/UX Overrun	Medium	Medium	Hard stop design polish at Hour 34; prioritize functionality over complex aesthetics.	F

6. Resource Plan

Personnel: 6 Members (3 Frontend, 2 Backend/Db 1 ML Overlap)

Tools & Technology:

- **Database:** Cloud Office Suite Tables (e.g., Google Sheets/Airtable/Excel Online).
- **Automation:** Cloud Office Scripting (e.g., Google Apps Script/Power Automate) or Lightweight Python Scripts (FastAPI layer).
- **Frontend:** HTML/Tailwind CSS/JS (One-file mandate).

- **Specialized Services:** Navneet's pre-integrated FastAPI microservice for ML and LLM endpoints.